

Household Energy Consumption Forecasting

Repository: Energy Consumption Forecasting using classical ML models (KNN, Random Forest, Gradient Boosting, XGBoost) with an interactive Streamlit deployment.

Table of Contents

1. [Project Overview](#)
 2. [Dataset](#)
 3. [Preprocessing and Feature Engineering](#)
 4. [Models and Hyperparameters](#)
 5. [Evaluation and Metrics](#)
 6. [Model Comparison](#)
 7. [Streamlit Application](#)
 8. [Repository Structure](#)
 9. [How to Run Locally](#)
 10. [Requirements](#)
 11. [Limitations and Future Work](#)
 12. [Live Demo](#)
 13. [Author](#)
 14. [License](#)
-

Project Overview

This project focuses on predicting household energy consumption using the Household Power Consumption dataset. The work involves data cleaning, preprocessing, feature engineering, model training, and evaluation using multiple machine learning techniques. A Streamlit application has also been developed to allow interactive forecasting and visualization of results.

Dataset

- **Source:** Household Power Consumption dataset.
 - **Size:** ~1,048,575 rows.
 - **Transformations Applied:**
 - Combined `Date` and `Time` columns into a unified `datetime` column.
 - Resampled the dataset at 30-minute intervals.
 - Aggregated sub-meter readings into `Laundry`, `Kitchen`, and `Appliances`.
 - Converted power measurements into kWh.
-

Preprocessing and Feature Engineering

1. **Missing Values:** Replaced '?' with NaN and handled missing data with forward fill and drop operations.
 2. **Resampling:** Converted data to 30-minute intervals with sum or average aggregations.
 3. **Feature Engineering:** Created time-based features (Hour, Weekday, Month, Is_weekend).
 4. **Lag and Rolling Features:** Added lag features (1-12), rolling means (3, 6, 12 steps), and lagged values for Voltage and Current.
 5. **Scaling:** Applied StandardScaler for KNN, while tree-based models did not require scaling.
 6. **Train-Test Split:** Time-aware splitting with shuffle=False to maintain chronological order.
-

Models and Hyperparameters

- **K-Nearest Neighbors (KNN):** n_neighbors=11, with scaled inputs.
 - **Random Forest Regressor:** n_estimators=200, max_depth=10, random_state=42.
 - **Gradient Boosting Regressor:** n_estimators=200, learning_rate=0.05, max_depth=6, random_state=42.
 - **XGBoost Regressor:** Tuned using Optuna with multiple trials, selecting the best parameters for performance.
-

Evaluation and Metrics

Evaluation metrics used:

- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R² Score (Coefficient of Determination)

Results from the Notebook:

Model	RMSE	MAE	R ²
KNN	0.1823	0.1153	0.8490
Random Forest	0.1617	0.0262	0.8811
Gradient Boosting	0.1401	0.0932	0.9024
XGBoost (Optuna)	0.1378	0.0857	0.9138

Naive Baseline: R² = 0.6387, RMSE = 0.2820, MAE = 0.1643.

XGBoost demonstrated the best performance with the lowest RMSE and the highest R².

Model Comparison

Relative improvements compared to the naive baseline: - **KNN**: RMSE ↓ ~35%, R^2 ↑ by ~21%.

- **Random Forest**: RMSE ↓ ~43%, R^2 ↑ by ~24%.

- **Gradient Boosting**: RMSE ↓ ~50%, R^2 ↑ by ~26%.

- **XGBoost (Optuna)**: RMSE ↓ ~51%, R^2 ↑ by ~27%.

Streamlit Application

The Streamlit application provides an interactive platform to forecast energy consumption: - Users can input **Hour, Weekday, Month**, and select a model.

- Predictions are generated for the next **3 intervals** using recursive forecasting.

- Outputs include both **numerical predictions** and a **line chart visualization**.

Repository Structure

```
├─ Energy_4.ipynb      # Main notebook (EDA, preprocessing, modeling,
evaluation)
├─ models/             # Saved trained models (.pkl)
├─ data/               # Dataset (CSV files)
├─ streamlit_app.py    # Streamlit deployment script
├─ requirements.txt    # Dependencies
└─ README.md           # Documentation
```

How to Run Locally

1. Clone this repository:

```
git clone https://github.com/yourusername/energy-forecasting.git
cd energy-forecasting
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Run the Streamlit application:

```
streamlit run streamlit_app.py
```

Requirements

```
pandas  
numpy  
matplotlib  
seaborn  
scikit-learn  
xgboost  
optuna  
joblib  
streamlit  
jupyterlab
```

Limitations and Future Work

- Incorporate external data such as weather and occupancy.
 - Explore direct multi-step forecasting techniques.
 - Add explainability tools (e.g., SHAP values) for better interpretability.
 - Experiment with deep learning models such as LSTM or Transformers.
-

Live Demo

The deployed application can be accessed here:

👉 [Streamlit App Link](#)

Author

Atish Sawant

Data Science and Machine Learning Enthusiast

- GitHub: [yourusername](#)
 - Email: [your.email@example.com](#)
-

License

This project is licensed under the MIT License.